CHARUSAT®

अमृतं तु विद्या

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Fundamentals Of Data Structure And Algorithms

Github Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/tree/main/Practical-4

Practical-4.1

Difficulty :

Understanding how nodes are connected using pointers. Inserting a new node at the front, end, and specific position was a little confusing. Handling an invalid position was also a challenge.

Error :

If the given position is greater than the length of the linked list, insertion cannot be performed.Incorrect pointer linking can cause the list to break.

Solution:

Used a Node structure with token and next. Used separate functions for insertion at front, end, and specific position. Checked whether the position is valid before inserting. Used display() to check the list after every operation.

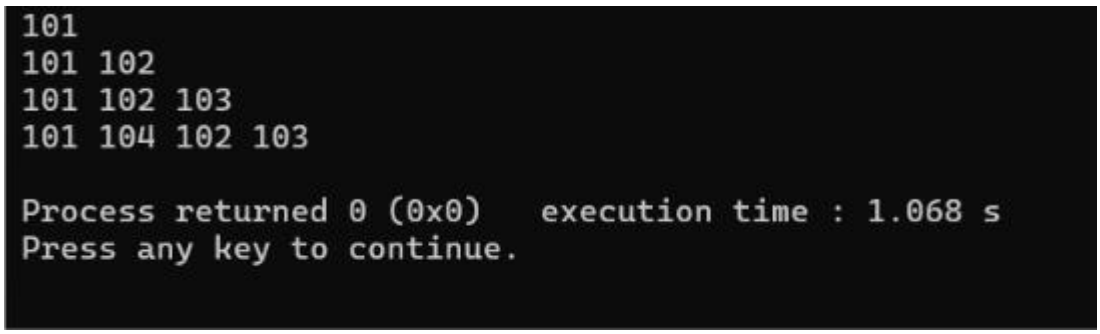
Outcome:

The program successfully performs insertion at the **front, end, and specific position** of a singly linked list and displays the updated queue after each operation.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-4/pr%204.1.cpp

Output:



Key Questions:

Problem-1

Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

Answer:

If the position is greater than the list length, the program shows an error message. It does not insert the node at the end. This makes the program safe and avoids wrong insertion. It assumes the user enters a valid position.

Practical-4.2

Difficulty:

Understanding how to delete a node by its token value. Printing the linked list from last to first was a little difficult. Understanding how to reverse-print without changing the original list.

Error :

The given token may not exist in the queue. Wrong pointer handling during deletion can break the linked list.

Solution:

Search for the token and delete that node using pointers. For reverse printing, use **recursion** to reach the last node first and then print while returning. The original queue is not changed.

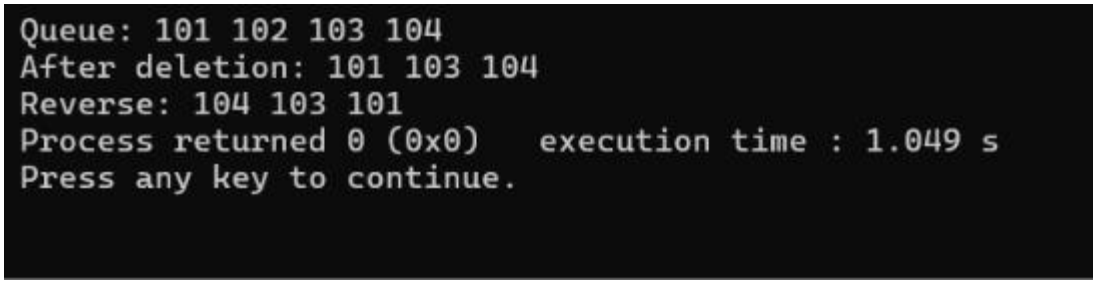
Outcome:

The program successfully deletes a patient token, prints the queue from front to back, and reverse-prints it from last to first.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-4/pr%204.2.cpp

Output:



Key Questions:

Problem-2

If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

Answer:

If the node is the only node, only the head pointer needs to change.The head is set to NULL, making the list empty.For a middle node, the previous node's next pointer is changed.The main difference is that the only node has no previous or next node.